

Facultad de ingeniería

Escuela de ciencias y sistemas

Vacaciones del primer semestre

Introducción a la programación y computación 2

Actividad 17/06

Gilberto Geovanni Hernández Asturias

202404410

Fundamentación Teórica y Análisis Crítico

La Limitación del Monolito Local: Cuando la interfaz de usuario, la lógica de negocio y el almacenamiento residen exclusivamente en una única máquina física aislada (monolito local), surgen importantes limitaciones estructurales que afectan la evolución del software.

Problemas de Sincronización: La ausencia de una base de datos centralizada accesible mediante red provoca que la información permanezca confinada a una única máquina. Como consecuencia, cuando varios usuarios requieren utilizar el sistema simultáneamente, mantener la consistencia de los datos en tiempo real se vuelve una tarea compleja o incluso imposible.

Problemas de Escalabilidad: Los recursos disponibles quedan restringidos al hardware de una sola computadora, incluyendo procesador, memoria RAM y almacenamiento. Bajo este modelo, el sistema presenta dificultades para soportar incrementos significativos en la cantidad de usuarios o transacciones concurrentes, imposibilitando además la escalabilidad horizontal mediante la incorporación de nuevos servidores.

Diferencia entre Layers y Tiers: Es común confundir ambos conceptos; sin embargo, representan niveles distintos de diseño arquitectónico. Los layers o capas lógicas corresponden a una organización conceptual del código fuente dentro de la aplicación. Su finalidad principal es separar responsabilidades específicas, como la presentación, la lógica de negocio y el acceso a datos, facilitando el mantenimiento y la evolución del software. Todas las capas pueden ejecutarse dentro de un mismo servidor o proceso sin necesidad de distribución física. Mientras que los Tiers representan la distribución real de los componentes en diferentes servidores, máquinas virtuales o infraestructuras independientes conectadas mediante una red.

Arquitectura de Tres Niveles (3-Tier Architecture):

Nivel 1: Capa de Presentación (Presentation Tier): Es responsable de la interacción con el usuario. Captura solicitudes y muestra resultados mediante interfaces gráficas.

Tecnologías comunes:

- HTML5
- CSS3
- JavaScript
- React

Nivel 2: Capa de Aplicación o Negocio (Application Tier): Constituye el núcleo funcional del sistema. Actúa como intermediario obligatorio entre la interfaz de usuario y la base de datos, aquí se implementan:

- Reglas de negocio
- Validaciones
- Procesamiento de datos
- Algoritmos
- Toma de decisiones

Nivel 3: Capa de Datos (Data Tier): Su función consiste en almacenar y administrar la información de manera persistente y segura. Tecnologías comunes:

- SQL Server
- PostgreSQL
- MySQL
- MongoDB

Seguridad Perimetral: Expone directamente una base de datos a Internet representando un riesgo crítico de seguridad informática, permitir acceso público a dichos servicios incrementa significativamente la superficie de ataque frente a:

- Ataques de fuerza bruta.
- Explotación automatizada de vulnerabilidades.
- Accesos no autorizados.

Buenas Prácticas: La arquitectura recomendada consiste en:

1. Mantener la base de datos dentro de una red privada.
2. Permitir conexiones únicamente desde el servidor de aplicaciones.
3. Implementar controles de acceso y segmentación de red.
4. Evitar cualquier comunicación directa entre usuarios externos y el servidor de base de datos.

Desacoplamiento Lógico con el Patrón MVC

La Crisis del Código Espagueti: La mezcla de código HTML, consultas SQL y lógica de negocio dentro de un mismo archivo genera sistemas difíciles de mantener. Entre las principales consecuencias se encuentran:

- Incremento de errores.
- Dificultad para depuración.
- Baja reutilización de código.

Separación de Responsabilidades (Separation of Concerns): El patrón MVC propone dividir el sistema en tres componentes claramente diferenciados.

Modelo (Model): Representa los datos y las reglas de negocio de la aplicación.

Responsabilidades principales:

- Gestionar entidades de dominio.
- Aplicar validaciones.
- Ejecutar reglas de negocio.
- Mantener independencia de la interfaz gráfica.

Vista (View): Es la responsable exclusiva de la presentación visual. Funciones:

- Mostrar información al usuario.
- Recibir datos preparados por el controlador.
- Generar interfaces visuales.

No debe contener:

- Consultas SQL.
- Reglas de negocio.
- Procesamiento complejo de datos.

Controlador (Controller): Actúa como intermediario entre la Vista y el Modelo.

Responsabilidades:

1. Recibir solicitudes HTTP.
2. Procesar parámetros de entrada.
3. Invocar lógica de negocio.
4. Obtener datos del Modelo.
5. Seleccionar la Vista adecuada.
6. Retornar la respuesta al usuario.

Métricas de Ingeniería de Software: En la alta cohesión cada componente se concentra en una única responsabilidad específica:

Componente	Responsabilidad
------------	-----------------

Modelo	Datos y lógica de negocio
--------	---------------------------

Vista	Presentación visual
-------	---------------------

Controlador	Coordinación y flujo
-------------	----------------------

Esta especialización favorece la calidad y mantenibilidad del sistema.

Bajo Acoplamiento: Los componentes minimizan sus dependencias directas.

Beneficios:

- Mayor facilidad de mantenimiento.
- Menor impacto ante cambios.
- Mejor reutilización de código.
- Mayor facilidad para realizar pruebas.

Modelado del Ciclo de Vida y Enrutamiento Semántico

Mapeo Analítico de URLs:

URL Entrante	Clase controladora	Acción Ejecutada	Parámetro ID
https://ingenieria.usac.edu.gt/ControlAcademico/Login	ControlAcademicoController	Login	Ninguno
https://ingenieria.usac.edu.gt/Estudiante/Historial/20260123	EstudianteController	Historial	20260123
https://ingenieria.usac.edu.gt/Asignacion/Detalle/10	AsignacionController	Detalle	10
https://ingenieria.usac.edu.gt/Home	HomeController	Index	Ninguno
https://ingenieria.usac.edu.gt/	HomeController	Index	Ninguno

Flujo de una Petición HTTP en MVC

Paso 1. Solicitud del Cliente: El usuario interactúa con la interfaz y genera una petición HTTP (GET o POST) que es enviada al servidor de aplicaciones.



Paso 2. Enrutamiento: El middleware de ASP.NET Core analiza la URL recibida y determina qué controlador y acción deben ejecutarse.



Paso 3. Ejecución del Controlador: El framework crea una instancia del controlador correspondiente y ejecuta el método asociado a la ruta solicitada.



Paso 4. Interacción con el Modelo: El controlador consulta al Modelo para obtener o procesar la información requerida.



Paso 5. Preparación de la Vista: Los datos retornados por el Modelo son enviados a la Vista mediante el mecanismo:

View(modelo)



Paso 6. Renderizado

La Vista combina la plantilla visual con los datos proporcionados y genera el documento HTML final.



Paso 7. Respuesta al Cliente

El servidor devuelve una respuesta HTTP al navegador, que interpreta y muestra el contenido al usuario.

Referencias Bibliográficas

Facultad de Ingeniería, USAC. (2026). *Sesión 11: Modelado Base y Arquitecturas de Despliegue. Evolución de Sistemas Distribuidos, Fundamentos del Modelo Cliente-Servidor y Diseño Físico Multi-Capas (N-Tier).* Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.

Facultad de Ingeniería, USAC. (2026). *Sesión 12: Arquitectura y Componentes del Patrón MVC. Desacoplamiento Lógico de Software, Ciclo de Vida de las Peticiones y Enrutamiento en Aplicaciones Interactivas Modernas.* Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.

Captura de pantalla:

localhost:7243/Estudiante/Listar

Control Académico - Listado Oficial

Carné	Nombre Completo	Promedio General
2026012	Fernando Velásquez	91.5
2026045	Maria Mercedes	84